Road Trip Route Planner – Project Report

CSCE 2110.501 – Foundations of Data Structures | Team 6: Yahya Baban and Emir Alakus

1. Introduction and design issues

The program models North Texas cities as vertices and highway corridors as edges and answers route questions in C++17: menu option 2 shortest route, option 3 route with strictly fewer than x connections, option 4 route through two required cities in either order, option 5 shortest round trip that visits every reachable city and ends at the first return to the start, and option 6 a meeting city that minimizes the combined number of connections of three travelers. “Shortest” means fewest connections, not miles or time. The graph is undirected and unweighted. Main design issues were (a) how detailed the map should be – we use one vertex per city and one edge per highway corridor so the project stays about graph algorithms; (b) what extra state the round trip needs; and (c) keeping routes explainable by storing a highway label for each edge.

2. Data and data structures

We created the input ourselves: cities.txt (15 cities, 19 roads, selected real corridors checked against NTTA/TxDOT pages listed in the README), cities_disconnected.txt (6 cities), cities_large.txt (60 cities), and random graphs from gen_graph.cpp for timing. A file has a header (cities, roads), the city names, and one road per line (two cities and a highway label). Everything is stored in one Graph class:

- vector<vector<int>> neighbors – adjacency list; chosen because the network is sparse (O(V + E) space instead of O(V^2) for a matrix).
- unordered_map<string,int> and vector<string> – name ↔ integer ID with O(1) average lookup.
- vector<unordered_map<int,string>> – highway label of each road, used to print every leg.
- queue<int>, distance and parent vectors – BFS frontier, number of connections, and route reconstruction.
- A parent array indexed by (visited-set bitmask, current city) – states of the round-trip search.

3. Algorithms and justification

Breadth-first search (BFS). All edges cost one connection, so exploring level by level reaches every city by a route with the fewest connections. The first visit to a city sets its distance and parent; following parents backward and reversing gives the route. Options 2, 3, 4 and 6 reuse this BFS (the connection limit compares the shortest length with x; required stops run BFS from the start and both stops, compare the two stop orders and join the shortest legs; the meeting city runs BFS from the three travelers and scans every other reachable city for the smallest sum of distances).

Round trip. A city alone does not describe a trip, so each BFS state is (set of visited cities as a bitmask, current city). A move along a road sets a bit; returning to the start is blocked until every reachable city is visited, which implements “once you come back, you do not leave again”. BFS explores states by number of moves, so the first complete return is optimal. Cities may repeat (it is not a Hamiltonian cycle). If the start is a cut city, such as Dallas in our map, no valid trip exists and the program reports it. This is a travelling-salesman-like problem and is NP-hard in general, so the exact search is limited to 18 reachable cities.

4. Asymptotic running time and space

V = number of cities, E = number of roads, K and M = cities and roads in the start's connected component, R = road lines in the file.

Operation	Time	Space
Options 2, 3, 4 and 6 (constant number of BFS runs)	O(V + E)	O(V) extra
Graph storage	–	O(V + E)
Option 5 (round trip)	O(V + E + 2^K (K + M))	O(V + K·2^K) extra
Loading the file (duplicate-road check)	O(V + R·V) worst case	O(V + E)

BFS touches each city and each edge entry a constant number of times, so it is linear. The round trip has 2^K·K states and each state scans its roads, which is exponential in K.

5. Optimization, scalability and testing

Optimizations: integer IDs instead of strings inside searches; one BFS per source reused for several answers; the round-trip search is limited to the start's component and stops as soon as the goal state is reached. Measured times (end to end, including loading, one Linux machine, -O2) are not Big O bounds: BFS with E = 3V took about 0.006 s (V = 1,000), 0.02 s (10,000), 0.44 s (100,000) and 7.2 s (1,000,000); file loading takes almost as long as the whole run. The round trip took 0.006–0.007 s for K = 10–14, 0.013 s for K = 16 and 0.06 s for K = 18, consistent with doubling states. We tested every option on normal and edge cases (strict limit, unknown city, disconnected graph, same start and end, no round trip from Dallas, repeated meeting cities, size limit). All 225 ordered city pairs matched an independent all-pairs shortest-path calculation, and the round-trip optimum (19 connections from Allen) was verified independently.

6. Remarks, limitations and future work

The model counts connections, not distance or time; names are case-sensitive and one word; the exact round trip supports up to 18 reachable cities. Future work: distance or time weights with Dijkstra's algorithm, a finer junction model, and an approximate round trip for larger graphs. AI assistance (ChatGPT and Claude) was used to help write and test code, the README, this report and the slides; we reviewed and ran everything. No code was copied from other projects; BFS and bitmask search are standard course techniques. References: course materials, NTTA, TxDOT and City of Richardson pages (URLs in README), cppreference.com.

Appendix: team contributions

Yahya Baban – 50%; Emir Alakus – 50%. We completed all parts together: design, coding, datasets, testing, benchmarks, README, report and slides.